```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
arr = np.array([1200, 2500, 1800, 3200, 4500, 900, 2100])

print("Array:", arr)

print("Dimensions:", arr.ndim)
print("Shape:", arr.shape)
print("Total elements:", arr.size)
print("Data type:", arr.dtype)

print("Sum:", np.sum(arr))
print("Average:", np.mean(arr))
print("Minimum:", np.min(arr))
print("Maximum:", np.max(arr))

increased_arr = arr * 1.10
print("Values increased by 10%:", increased_arr)

filtered_arr = arr[arr > 2000]
print("Values greater than 2000:", filtered_arr)
```

```
Array: [1200 2500 1800 3200 4500  900 2100]
Dimensions: 1
Shape: (7,)
Total elements: 7
Data type: int64
Sum: 16200
Average: 2314.285714285714
Minimum: 900
Maximum: 4500
Values increased by 10%: [1320. 2750. 1980. 3520. 4950.  990. 2310.]
Values greater than 2000: [2500 3200 4500 2100]
```

```
arr2 = np.array([
    [1200, 2500, 1800],
    [3200, 4500, 900],
    [2100, 2800, 3500]
])

print("2D Array:\n", arr2)

print("Dimensions:", arr2.ndim)
print("Shape:", arr2.shape)
print("Total elements:", arr2.size)
print("Data type:", arr2.dtype)
print("Sum of all elements:", np.sum(arr2))
print("Sum of each row:", np.sum(arr2, axis=1))
print("Sum of each column:", np.sum(arr2, axis=0))
print("Average:", np.mean(arr2))
print("Minimum:", np.min(arr2))
print("Maximum:", np.max(arr2))

print("Element at row 1, column 2:", arr2[0, 1])
print("First row:", arr2[0, :])
print("Second row:", arr2[1, :])
print("First column:", arr2[:, 0])
print("First two rows:\n", arr2[:2, :])
```

```
2D Array:
 [[1200 2500 1800]
 [3200 4500  900]
 [2100 2800 3500]]
Dimensions: 2
Shape: (3, 3)
Total elements: 9
Data type: int64
Sum of all elements: 22500
Sum of each row: [5500 8600 8400]
Sum of each column: [6500 9800 6200]
Average: 2500.0
Minimum: 900
Maximum: 4500
Element at row 1, column 2: 2500
First row: [1200 2500 1800]
Second row: [3200 4500  900]
First column: [1200 3200 2100]
First two rows:
 [[1200 2500 1800]
 [3200 4500  900]]
```

```
from google.colab import files

uploaded = files.upload()
```

Choose Files Online_Retail.csv

Online_Retail.csv(text/csv) - 47217062 bytes, last modified: 10/29/2026 - 100% done

```
Saving Online_Retail.csv to Online_Retail (1).csv
```

```
filename = next(iter(uploaded))
df = pd.read_csv("Online_Retail.csv")

display(df.head())

print(df.columns.tolist())

print("Rows and columns:", df.shape)
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	Customer
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/01/2010 08:26:00	2.55	1785
1	536365	71053	WHITE METAL LANTERN	6	12/01/2010 08:26:00	3.39	1785
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/01/2010 08:26:00	2.75	1785

```
df["Revenue"] = df["Quantity"] * df["UnitPrice"]

display(df.head())
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	Customer
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/01/2010 08:26:00	2.55	1785
1	536365	71053	WHITE METAL LANTERN	6	12/01/2010 08:26:00	3.39	1785
2	536365	84406B	CREAM CUPID HEARTS COAT	8	12/01/2010 08:26:00	2.75	1785

```
print("Total quantity:", df["Quantity"].sum())
print("Average quantity:", df["Quantity"].mean())
print("Minimum quantity:", df["Quantity"].min())
print("Maximum quantity:", df["Quantity"].max())
print("Number of quantity records:", df["Quantity"].count())

print("\nTotal revenue:", df["Revenue"].sum())
print("Average revenue per row:", df["Revenue"].mean())
print("Minimum revenue:", df["Revenue"].min())
print("Maximum revenue:", df["Revenue"].max())

print("\nAggregate statistics for numeric columns:")
display(df.agg({
    "Quantity": ["sum", "mean", "min", "max", "count"],
    "UnitPrice": ["sum", "mean", "min", "max", "count"],
    "Revenue": ["sum", "mean", "min", "max", "count"]
}))
```

```
Total quantity: 5176450
Average quantity: 9.55224954743324
Minimum quantity: -80995
Maximum quantity: 80995
Number of quantity records: 541909

Total revenue: 9747747.933999998
Average revenue per row: 17.98779487699964
Minimum revenue: -168469.6
Maximum revenue: 168469.6

Aggregate statistics for numeric columns:
```

	Quantity	UnitPrice	Revenue
sum	5.176450e+06	2.498804e+06	9.747748e+06
mean	9.552250e+00	4.611114e+00	1.798779e+01
min	-8.099500e+04	-1.106206e+04	-1.684696e+05
max	8.099500e+04	3.897000e+04	1.684696e+05
count	5.419090e+05	5.419090e+05	5.419090e+05

```
# Rows where revenue is greater than 2000
high_revenue = df[df["Revenue"] > 2000]

print("Rows with revenue greater than 2000:")
display(high_revenue.head(10))

# Rows where quantity is greater than 5
high_quantity = df[df["Quantity"] > 5]

print("\nRows where quantity is greater than 5:")
display(high_quantity.head(10))

# Compare rows with positive and negative quantities
print("\nPositive quantity rows:", (df["Quantity"] > 0).sum())
print("Negative quantity rows:", (df["Quantity"] < 0).sum())

# Compare rows with positive and negative prices
print("Positive price rows:", (df["UnitPrice"] > 0).sum())
print("Zero or negative price rows:", (df["UnitPrice"] <= 0).sum())
```

```
Rows with revenue greater than 2000:
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice
15017	537632	AMAZONFEE	AMAZON FEE	1	12/07/2010 15:08:00	13541.33
16427	537657	22189	CREAM HEART CARD HOLDER	972	12/07/2010 16:42:00	2.31
16428	537657	22188	BLACK HEART CARD	972	12/07/2010 16:42:00	2.31

```python
# Quantity greater than 2 AND unit price greater than 100
filtered_data = df[
    (df["Quantity"] > 2) &
    (df["UnitPrice"] > 100)
]

print("Records matching both conditions:")
display(filtered_data.head(10))

# Filter by country
country_data = df[df["Country"] == "United Kingdom"]

print("\nUnited Kingdom records:")
display(country_data.head(10))

# Filter using a country and revenue condition
country_revenue = df[
    (df["Country"] == "United Kingdom") &
    (df["Revenue"] > 2000)
]

print("\nUK records with revenue greater than 2000:")
display(country_revenue.head(10))
```

			DRAWERS			
19871	537899	22328	ROUND SNACK BOXES SET OF 4 FRUITS	1488	12/09/2010 10:44:00	2.55

```
Rows where quantity is greater than 5:
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	Custome
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/01/2010 08:26:00	2.55	1785
1	536365	71053	WHITE METAL	6	12/01/2010 08:26:00	3.39	1785

			LANTERN				
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/01/2010 08:26:00	2.75	178
3	536365	84029G	KNITTED UNION FLAG HOT WATER	6	12/01/2010 08:26:00	3.39	178

```
Records matching both conditions:
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	Cus
61549	541426	22823	CHEST NATURAL WOOD 20 DRAWERS	4	01/18/2011 08:44:00	110.0	
66285	541789	22823	CHEST NATURAL WOOD 20 DRAWERS	3	01/21/2011 13:07:00	110.0	
222680	556444	22502	PICNIC BASKET WICKER 60 PIECES	60	06/10/2011 15:28:00	649.5	
423769	573153	POST	POSTAGE	3	10/28/2011 07:39:00	145.0	
468546	576512	22823	CHEST NATURAL WOOD 20 DRAWERS	4	11/15/2011 12:09:00	110.0	

```python
conditions = [
    df["Revenue"] < 100,
    (df["Revenue"] >= 100) & (df["Revenue"] <= 1000),
    df["Revenue"] > 1000
]

categories = ["Low", "Medium", "High"]

df["RevenueCategory"] = np.select(
    conditions,
    categories,
    default="Unknown"
)

print("Revenue categories:")
display(df[["Quantity", "UnitPrice", "Revenue", "RevenueCategory"]].head(10))

print("\nCount of records in each category:")
print(df["RevenueCategory"].value_counts())
```

3	536365	84029G	FLAG HOT WATER BOTTLE	6	08:26:00	3.39	1785
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/01/2010 08:26:00	3.39	1785

```
Revenue categories:
```

	Quantity	UnitPrice	Revenue	RevenueCategory
0	6	2.55	15.30	Low
1	6	3.39	20.34	Low
2	8	2.75	22.00	Low
3	6	3.39	20.34	Low
4	6	3.39	20.34	Low
5	2	7.65	15.30	Low
6	6	4.25	25.50	Low
7	6	1.85	11.10	Low
8	6	1.85	11.10	Low
9	32	1.69	54.08	Low

5	536365	22752	SET 7 BABUSHKA NESTING	2	12/01/2010 26:00	7.65	1785
			GLASS STAR HOLDER		12/01/2010 08:26:00	4.25	1785
7	536366	22633	UNION	6	12/01/2010 08:28:00	1.85	1785
8	536366	22632	HAND WARMER	6	12/01/2010 08:28:00	1.85	1785
9	536367	84879	ASSORTED COLOUR BIRD ORNAMENT	32	12/01/2010 08:34:00	1.69	1304

```
Count of records in each category:
RevenueCategory
Low       528334
Medium     13187
High         388
Name: count, dtype: int64
```

```
df["InvoiceDate"] = pd.to_datetime(
    df["InvoiceDate"],
    errors="coerce"
)

print("Date column converted successfully!")

print("InvoiceDate data type:", df["InvoiceDate"].dtype)

display(df[["InvoiceDate"]].head())

print("Invalid or missing dates:", df["InvoiceDate"].isna().sum())
```

```
Date column converted successfully!
InvoiceDate data type: datetime64[ns]
```

	InvoiceDate
0	2010-12-01 08:26:00
1	2010-12-01 08:26:00
2	2010-12-01 08:26:00
3	2010-12-01 08:26:00
4	2010-12-01 08:26:00

```
Invalid or missing dates: 0
```

```python
df["Year"] = df["InvoiceDate"].dt.year
df["Month"] = df["InvoiceDate"].dt.month
df["Day"] = df["InvoiceDate"].dt.day
df["MonthName"] = df["InvoiceDate"].dt.month_name()
df["DayOfWeek"] = df["InvoiceDate"].dt.day_name()

print("New date columns created:")
display(
    df[["InvoiceDate", "Year", "Month", "MonthName", "Day", "DayOfWeek"]].head(
)
```

```
New date columns created:
```

	InvoiceDate	Year	Month	MonthName	Day	DayOfWeek
0	2010-12-01 08:26:00	2010	12	December	1	Wednesday
1	2010-12-01 08:26:00	2010	12	December	1	Wednesday
2	2010-12-01 08:26:00	2010	12	December	1	Wednesday
3	2010-12-01 08:26:00	2010	12	December	1	Wednesday
4	2010-12-01 08:26:00	2010	12	December	1	Wednesday
5	2010-12-01 08:26:00	2010	12	December	1	Wednesday
6	2010-12-01 08:26:00	2010	12	December	1	Wednesday
7	2010-12-01 08:28:00	2010	12	December	1	Wednesday
8	2010-12-01 08:28:00	2010	12	December	1	Wednesday
9	2010-12-01 08:34:00	2010	12	December	1	Wednesday

```python
revenue_by_country = (
    df.groupby("Country")["Revenue"]
    .sum()
    .sort_values(ascending=False)
)
```

```
print("Top 10 countries by revenue:")
display(revenue_by_country.head(10))

if "Description" in df.columns:
    revenue_by_product = (
        df.groupby("Description")["Revenue"]
        .sum()
        .sort_values(ascending=False)
    )

    print("\nTop 10 products by revenue:")
    display(revenue_by_product.head(10))

monthly_revenue = (
    df.dropna(subset=["InvoiceDate"])
    .groupby(df["InvoiceDate"].dt.to_period("M"))["Revenue"]
    .sum()
)

monthly_revenue.index = monthly_revenue.index.astype(str)

print("\nMonthly revenue:")
display(monthly_revenue)
```

```
Top 10 countries by revenue:
```

Country	Revenue
United Kingdom	8187806.364
Netherlands	284661.540
EIRE	263276.820
Germany	221698.210
France	197403.900
Australia	137077.270
Switzerland	56385.350
Spain	54774.580
Belgium	40910.960
Sweden	36595.910

dtype: float64

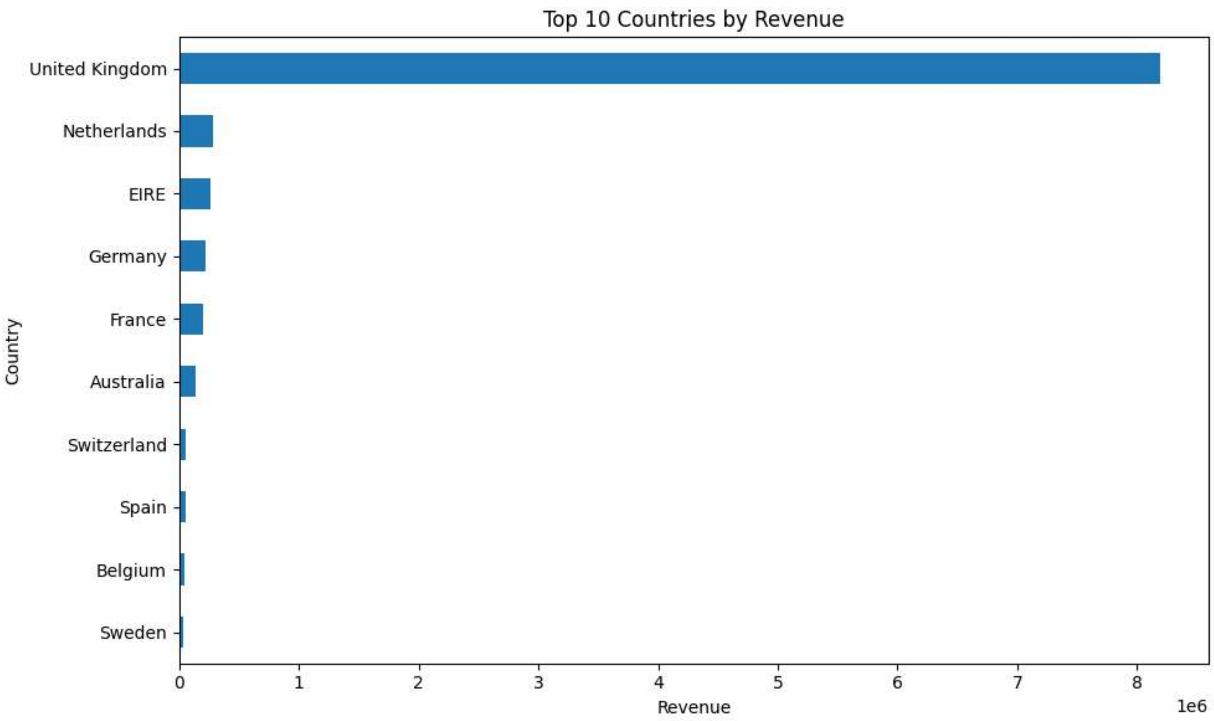
```python
top_countries = revenue_by_country.head(10).sort_values()

top_countries.plot(kind="barh", figsize=(10, 6))
plt.title("Top 10 Countries by Revenue")
plt.xlabel("Revenue")
plt.ylabel("Country")
plt.tight_layout()
plt.show()
```

PARTY BUNTING	98302.98
JUMBO BAG RED RETROSPOT	92356.03
RABBIT NIGHT LIGHT	66756.59
POSTAGE	66230.64
PAPER CHAIN KIT 50'S CHRISTMAS	63791.94
ASSORTED COLOUR BIRD ORNAMENT	58959.73
CHILLI LIGHTS	53768.06

dtype: float64

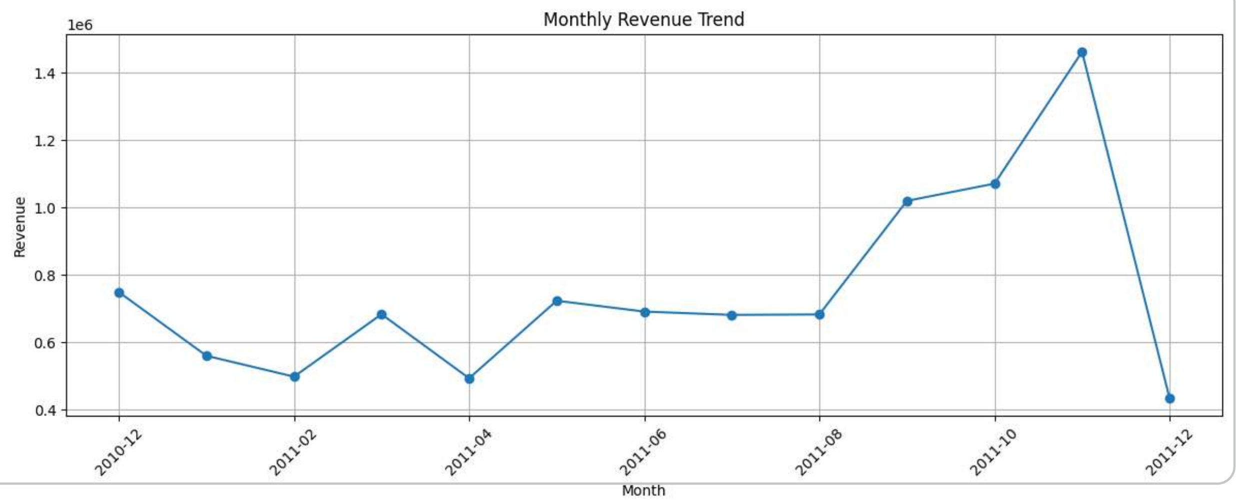
```
Monthly revenue:
```

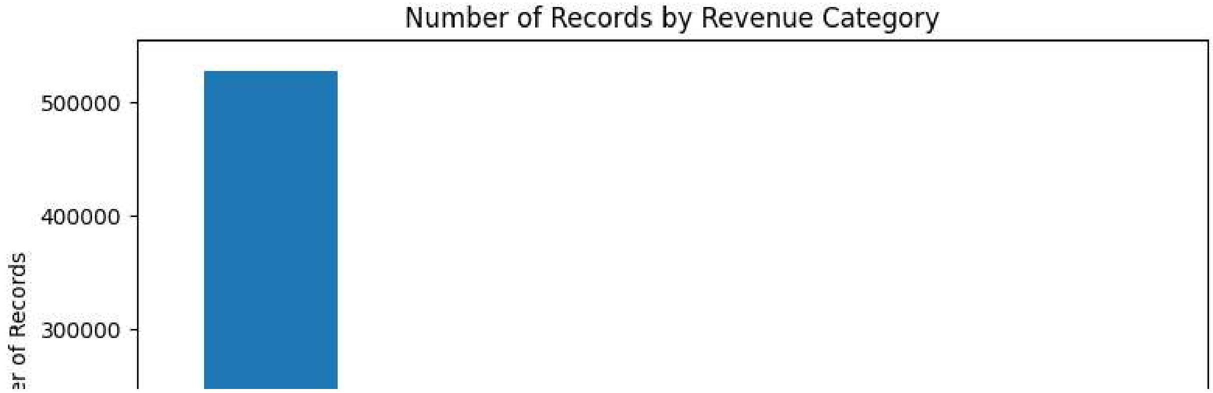
InvoiceDate	Revenue



```
2011-12        433668.010

dtype: float64
```

```
monthly_revenue.plot(kind="line", marker="o", figsize=(12, 5))
plt.title("Monthly Revenue Trend")
plt.xlabel("Month")
plt.ylabel("Revenue")
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.show()
```



```
df["RevenueCategory"].value_counts().reindex(
    ["Low", "Medium", "High", "Unknown"],
    fill_value=0
).plot(kind="bar", figsize=(8, 5))

plt.title("Number of Records by Revenue Category")
plt.xlabel("Revenue Category")
plt.ylabel("Number of Records")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```